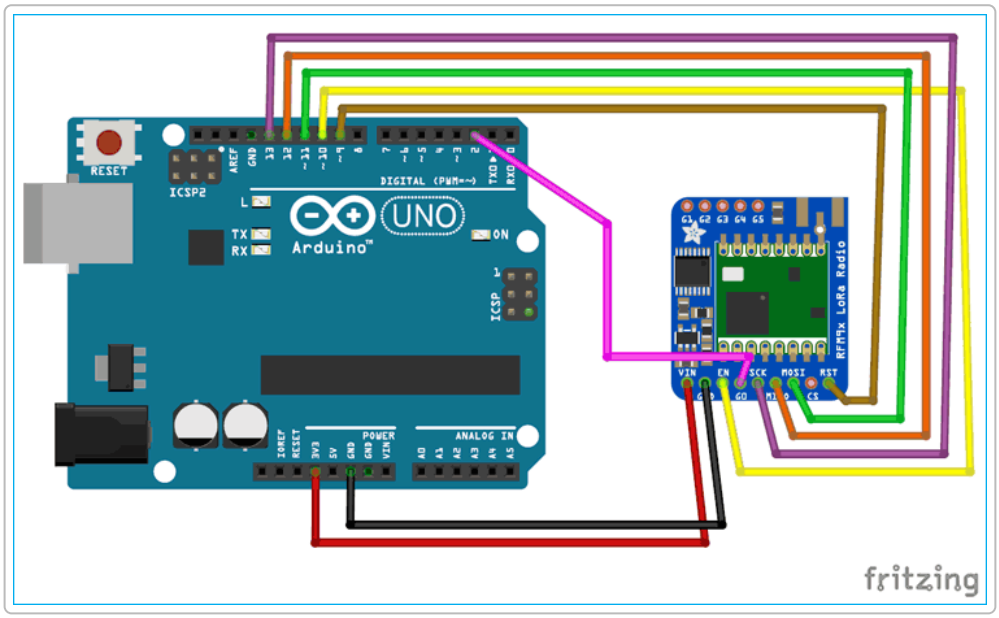
ChatGPT

LoRa-Based Boat Flip Detection System (Arduino Uno)

Goal: Set up two Arduino Uno–based boats with LoRa radios (SX1278 modules) and MPU6050 sensors so that when one boat **capsizes (flips over)**, it sends a LoRa alert to the other. We'll cover wiring, sensor orientation detection, unique IDs, LoRa communication, and provide example code for both boats.

Step 1: Hardware Setup – Wiring LoRa Module and MPU6050

LoRa Module (SX1278) Wiring: The SX1278 (Ra-02) LoRa module uses SPI to communicate with the Arduino. It must be powered at **3.3V** (***never 5V!***), or you risk destroying the module [1]. Connect the module to the Uno as follows (see image below):

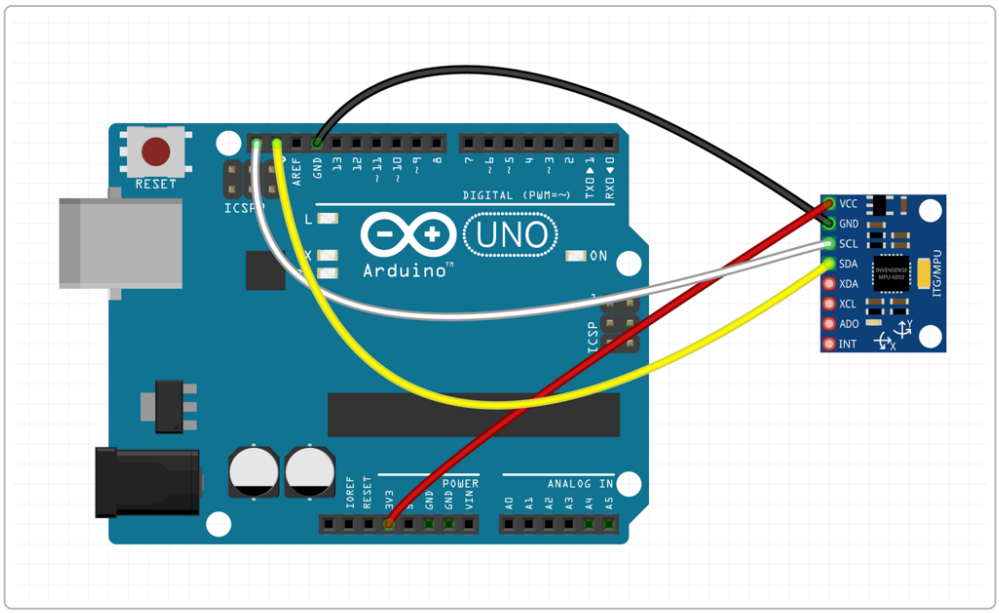


Wiring diagram: Arduino Uno connected to an SX1278 (Ra-02) LoRa module. The LoRa module's NSS (CS), MOSI, MISO, SCK, DIO0, RST, 3.3V, and GND pins are wired to the corresponding Uno pins as listed.

- **VCC** → Uno 3.3V (NOT 5V) [1]
- **GND** → Uno GND
- **NSS (CS)** → Uno D10 [2]
- **MOSI** → Uno D11 (UNO's SPI MOSI) [3]
- **MISO** → Uno D12 (UNO's SPI MISO) [3]
- **SCK** → Uno D13 (UNO's SPI SCK) [2]
- **DIO0 (G0)** → Uno D2 (used as interrupt/DIO0 pin) [2]
- **RESET** → Uno D9 [4]

Note: The Uno operates at 5V logic, while the SX1278 is a 3.3V device. In practice many have connected them directly, but it's safer to use logic level shifters or a 3.3V microcontroller to avoid damage [5]. Also, always attach a proper LoRa **antenna** to the module before transmitting (operating the radio without an antenna can damage it) [6].

MPU6050 Sensor Wiring: The MPU6050 IMU uses I²C for communication, so wiring is straightforward [7] (see second image below):



Wiring diagram: Arduino Uno connected to an MPU6050 (GY-521 breakout). The sensor's VCC, GND, SDA, and SCL pins connect to the Uno.

- **VCC** → Uno 5V (the common GY-521 module has an onboard 3.3V regulator, so 5V is acceptable; otherwise use 3.3V) [7]
- **GND** → Uno GND [7]
- **SDA** → Uno SDA (Arduino Uno's SDA pin is A4) [8]
- **SCL** → Uno SCL (Arduino Uno's SCL pin is A5) [8]

(The MPU6050's I²C address will be 0x68 by default. No additional connections are required unless you want to use its INT pin for interrupts, which isn't needed in our simple design.)

Step 2: Initializing LoRa Communication in the Arduino Sketch

We will use the Arduino LoRa library by Sandeep Mistry (install via Library Manager) [9] for easy packet radio communication. In your Arduino code, include the LoRa and SPI libraries, then initialize the LoRa module in `setup()`.

Key initialization steps: - Begin Serial (for debugging). - Set the LoRa module pins (if using the wiring above, the default `LoRa.setPins(SS, RST, DIO0)` are SS=D10, RST=D9, DIO0=D2, which match our connections). This step can be skipped if you used the default pins, but it's good practice to specify them. -

Call `LoRa.begin(frequency)` with the module's frequency (e.g. **433E6** for 433 MHz) [10]. This returns `true` if the radio initializes. If it fails, print an error and halt (likely a wiring issue).

Example – LoRa Init Code (in setup):

```
#include <SPI.h>
#include <LoRa.h>

#define LORA_CS   10    // LoRa NSS pin
#define LORA_RST  9     // LoRa RESET pin
#define LORA_DIO0 2     // LoRa DIO0 pin

void setup() {
  Serial.begin(9600);
  while (!Serial);  // wait for Serial monitor

  // Initialize LoRa radio
  LoRa.setPins(LORA_CS, LORA_RST, LORA_DIO0);
  if (!LoRa.begin(433E6)) {                // use 433E6 for 433 MHz SX1278 [10]
    Serial.println("LoRa init failed! Check wiring.");
    while (1);
  }
  Serial.println("LoRa init OK");
}
```

In this code, we configure the LoRa transceiver and set it to **433 MHz**. Make sure to use the frequency that matches your module and local regulations (SX1278 modules are typically 433 MHz by default). The LoRa library handles the SPI communication and configuration for us. If initialization succeeds, the radio is ready to send/receive packets. (If you have a different LoRa module frequency, adjust the `LoRa.begin()` call accordingly, e.g. 868E6 or 915E6.)

Step 3: Reading the MPU6050 for Boat Orientation

Each loop, the Arduino will read the MPU6050 to determine if the boat has flipped. The MPU6050 provides raw accelerometer (and gyro) data. We can use the accelerometer's readings to compute the device's tilt (roll and pitch angles) relative to gravity [11] [12]. When the boat is upright, gravity points mostly along the sensor's +Z axis; if the boat is upside down, the accelerometer's Z reading will reverse in sign (approximately –1g instead of +1g).

Initialize the MPU6050: Before reading, wake the sensor from sleep. In `setup()` (after `Wire.begin()`), write 0 to the MPU6050's power management register (0x6B) to ensure it's active:

```
#include <Wire.h>
const uint8_t MPU_ADDR = 0x68;  // I2C address of MPU6050
```

```
void setup() {
  // ... (LoRa init as above)
  Wire.begin();
  // Wake up MPU6050:
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(0x6B);    // PWR_MGMT_1 register
  Wire.write(0);       // set to zero (wakes up the MPU-6050)
  Wire.endTransmission(true);
}
```

Read accelerometer data: In the main loop, request the accelerometer registers (0x3B to 0x40). The MPU6050 outputs Ax, Ay, Az as 16-bit signed values. We'll retrieve and convert them to **g** units (by dividing by 16384, since the default sensitivity is ±2g). Then we calculate roll and pitch angles from these values:

```
void loop() {
  // Request 6 bytes of accelerometer data (Ax, Ay, Az)
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(0x3B);                      // starting register for accel data
  Wire.endTransmission(false);
  Wire.requestFrom(MPU_ADDR, 6, true);    // request 6 bytes
  if (Wire.available() == 6) {
    int16_t rawAx = Wire.read() << 8 | Wire.read();  // 16-bit Ax
    int16_t rawAy = Wire.read() << 8 | Wire.read();  // 16-bit Ay
    int16_t rawAz = Wire.read() << 8 | Wire.read();  // 16-bit Az

    // Convert to gravitational acceleration (g's):
    float Ax = rawAx / 16384.0;
    float Ay = rawAy / 16384.0;
    float Az = rawAz / 16384.0;

    // Compute tilt angles from accel data:
    float roll  = atan2(Ay, Az) * 180.0 / PI;                      // rotation
around X-axis 12
    float pitch = atan2(-Ax, sqrt(Ay*Ay + Az*Az)) * 180.0 / PI;     // rotation
around Y-axis 12

    // ... (we will use roll/pitch below to detect flips)
  }
}
```

Here we used the accelerometer-only method to get orientation: **roll** is `atan2(Y, Z)` and **pitch** is `atan2(-X, √(Y²+Z²))`, converting to degrees [12]. These formulas assume the sensor axes are aligned with the boat: Z axis roughly up/down, X axis forward/back, Y axis side-to-side. (The negative sign in pitch formula accounts for the definition of pitch angle increasing when the device tilts forward.)

Basic idea: When the boat is level, roll ≈ 0°, pitch ≈ 0°. If it's upside down, one of these angles will approach 180° or –180° (because the orientation is inverted). We'll use this fact to detect a flip.

Step 4: Detecting a "Flip" (Capsize) via Angle Threshold

To decide if the boat has flipped, we apply a threshold to the orientation angles. A simple rule: if the absolute **roll** or **pitch** angle exceeds about **90°**, the boat is essentially on its side or inverted (capsized). You can adjust this threshold, but 90° is a reasonable trigger point for a flip event.

Using the `roll` and `pitch` values from the accelerometer:

```
// ... inside loop, after computing roll and pitch:
bool flipped = false;
if (fabs(roll) > 90.0 || fabs(pitch) > 90.0) {
  flipped = true;
  Serial.println("Boat has flipped!");   // debug print
} else {
  flipped = false;
}
```

In the above logic, whenever `flipped` becomes `true`, it means the boat's orientation is past horizontal – likely upside down or capsized.

Alternative method: Instead of calculating angles, you could simply check the accelerometer's Z-axis reading. For example, if **Az** (in units of g) drops below 0 (or a certain negative threshold like –0.8g), it indicates the board is upside down (gravity is now pulling in the opposite direction on Z). This is a quicker check, but using roll/pitch angles is more intuitive for understanding and accommodates any orientation (e.g., flipped sideways or end-over).

(You may also want to add a small delay or filtering to avoid transient false triggers – e.g., require the flipped condition to persist for a short time – but we will keep it simple.)

Step 5: Transmitting a LoRa Alert Packet When a Flip is Detected

When a boat flips, we need to send an alert via LoRa to the other boat. We will send a **small packet** containing the boat's ID and an event code. A straightforward format is a text string like: **"ID<boat_number>:FLIP"**. For example, if boat 2 flips, it might send the string `"ID2:FLIP"`. This way the receiving side knows *which* boat sent the alert and what happened.

In code, once we detect `flipped == true`, we use the LoRa library to transmit a packet:

```
if (flipped) {
  // Send LoRa packet to signal the flip
  LoRa.beginPacket();
```

```
    LoRa.print("ID");
    LoRa.print(BOAT_ID);        // include this boat's ID in the message
    LoRa.print(":FLIP");
    LoRa.endPacket();           // packet is transmitted [13]

    Serial.println("Flip alert sent via LoRa!");
  }
```

Explanation: We open a LoRa packet, print the components of our message (`"ID"` prefix, the boat's ID number, and the text `"FLIP"`), then end the packet to actually send it. The LoRa library handles the rest (packetization and RF transmission) [13].

Avoiding repeat messages: If the loop runs continuously, the above code would attempt to send on **every iteration** while the boat remains flipped. It's usually best to avoid spamming the same alert repeatedly. We can handle this by, for example, sending the alert only on the **transition** to flipped. One simple way is to use a static or global flag to remember if we've already sent a flip message and not send again until the boat has righted (see final code). For clarity, the snippet above shows the core sending mechanism without that guard.

Step 6: Receiving LoRa Packets and Identifying the Sender

On the other boat, we continuously listen for LoRa packets. Using `LoRa.parsePacket()`, we can check if a new packet has arrived [14]. If so, we read the packet's content and parse the message.

Receiver logic in loop:

```
  int packetSize = LoRa.parsePacket();
  if (packetSize) {
    // we have received some data
    String packet = "";
    while (LoRa.available()) {
      char c = (char) LoRa.read();
      packet += c;
    }
    Serial.print("Received packet: ");
    Serial.println(packet);

    // Check if it's a flip alert
    if (packet.indexOf("FLIP") != -1) {
      // Extract the sender's ID (expect format "IDx:FLIP")
      int idIndex = packet.indexOf("ID");
      char senderID = packet.charAt(idIndex + 2);  // character after "ID"
      Serial.print("**ALERT:** Boat ");
      Serial.print(senderID);
      Serial.println(" has flipped!");
      // (Here you could trigger an alarm, light an LED, etc.)
```

```
    }
  }
```

In this code, when a packet is received, we assemble it into a string (for example, we might get `"ID2:FLIP"`). We then check if the string contains the substring `"FLIP"`. If yes, we assume it's our flip alert format. We find the `"ID"` in the string and take the character immediately after it as the sender's ID. For simplicity, this assumes a single-digit ID. We then print a message or take action (in a real scenario, you might sound a buzzer or log the event).

This approach is easy to understand. Alternatively, one could send a binary packet (e.g., one byte for ID and one byte for event code) to avoid parsing strings. For example, send `[boatID, 0x01]` where `0x01` means "flipped." The receiver would read two bytes and interpret them. Both methods work; the text format is just human-readable for debugging.

(Note: `LoRa.parsePacket()` returns the length of the packet if one is available. We use `LoRa.read()` in a loop to retrieve all bytes [14]. Also, `LoRa.packetRssi()` can give signal strength if needed.)

Step 7: Setting Unique Boat IDs

We need each boat to have a unique identifier so we know who sent the alert. There are a few simple ways to assign an ID:

- **Hard-code it in software:** E.g., in the code use a constant `BOAT_ID` set to `1` for the first boat and `2` for the second. This is very easy – just remember to compile/upload the code separately with the different ID for each boat.
- **Hardware jumper or DIP switch:** Use an input pin that is wired differently on each boat to select the ID at runtime. For example, tie a particular Arduino pin to GND on Boat 2 but leave it HIGH on Boat 1, and have the code read that pin in setup to decide the ID number.

For simplicity, we'll demonstrate the **hardcoded approach** using a `#define` or constant. At the top of your sketch, define the boat's ID:

```
#define BOAT_ID  1  // Use 1 for one boat, 2 for the other (unique IDs)
```

When you program the second boat, change this to `2`. The code then uses `BOAT_ID` in the LoRa message (`LoRa.print(BOAT_ID)`) so the receiver sees the correct sender ID.

If you prefer a hardware method: set up a pin as input with pull-up, and connect it to ground on one of the boards. For instance:

```
pinMode(7, INPUT_PULLUP);
int boatID = digitalRead(7) ? 1 : 2;
```

This would read as `boatID = 1` if the pin is high (not grounded) or `2` if the pin is grounded. Then use `boatID` in your LoRa message. This way the same code binary could run on both boats, and the physical jumper determines the ID. Choose whichever method is easiest for you.

Step 8: Complete Example Code for the Boat Nodes

Below is a **combined sketch** incorporating all the above steps. Each boat will run the *same code*, with only the `BOAT_ID` constant differing. After programming, Boat #1 (e.g., ID=1) and Boat #2 (ID=2) will be able to communicate: if one detects a flip, it sends an alert packet that the other receives and processes.

```
#include <Wire.h>
#include <SPI.h>
#include <LoRa.h>

#define BOAT_ID    1             // <<<--- SET UNIQUE ID FOR EACH BOAT (1 or 2)
#define LORA_CS    10            // LoRa module CS pin
#define LORA_RST   9             // LoRa module RESET pin
#define LORA_DIO0  2             // LoRa module DIO0 pin

#define RAD_TO_DEG 57.2957795    // conversion factor (radians -> degrees)

const uint8_t MPU_ADDR = 0x68;   // I2C address of MPU6050 (0x68 by default)

bool flipSent = false;           // flag to indicate if flip alert was sent

void setup() {
  Serial.begin(9600);
  while (!Serial);  // wait for serial console (optional)

  // Initialize I2C (MPU6050) and wake sensor
  Wire.begin();
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(0x6B);          // PWR_MGMT_1 register
  Wire.write(0);             // set to 0 -> wakes up MPU6050
  Wire.endTransmission(true);

  // Initialize LoRa radio
  LoRa.setPins(LORA_CS, LORA_RST, LORA_DIO0);
  if (!LoRa.begin(433E6)) {   // initialize at 433 MHz
    Serial.println("LoRa init failed!");
    while (1);
  }
  Serial.println("LoRa init succeeded.");
  Serial.print("This is Boat ID=");
  Serial.println(BOAT_ID);
}
```

```
void loop() {
  // 1. Read accelerometer data from MPU6050
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(0x3B);                        // starting register for
accelerometer
  Wire.endTransmission(false);
  Wire.requestFrom(MPU_ADDR, 6, true);
  int16_t rawAx=0, rawAy=0, rawAz=0;
  if (Wire.available() == 6) {
    rawAx = Wire.read() << 8 | Wire.read();
    rawAy = Wire.read() << 8 | Wire.read();
    rawAz = Wire.read() << 8 | Wire.read();
  }

  // Convert raw accelerometer to G units
  float Ax = rawAx / 16384.0;
  float Ay = rawAy / 16384.0;
  float Az = rawAz / 16384.0;

  // Calculate pitch and roll (in degrees)
  float roll  = atan2(Ay, Az) * RAD_TO_DEG;
  float pitch = atan2(-Ax, sqrt(Ay*Ay + Az*Az)) * RAD_TO_DEG;

  // 2. Detect flip based on angle threshold
  bool flipped = false;
  if (fabs(roll) > 90.0 || fabs(pitch) > 90.0) {
    flipped = true;
  } else {
    flipped = false;
  }

  // 3. If flipped and not already sent, send LoRa alert
  if (flipped && !flipSent) {
    // Build packet: "ID<BOAT_ID>:FLIP"
    LoRa.beginPacket();
    LoRa.print("ID");
    LoRa.print(BOAT_ID);
    LoRa.print(":FLIP");
    LoRa.endPacket();
    Serial.println("** Sent FLIP ALERT via LoRa **");
    flipSent = true;
  }
  // Reset the sent flag if boat has righted back (so it can send again if it
flips again)
  if (!flipped && flipSent) {
    flipSent = false;
  }
```

```
  // 4. Listen for incoming LoRa packets
  int packetSize = LoRa.parsePacket();
  if (packetSize) {
    // Read the incoming packet
    String packet = "";
    while (LoRa.available()) {
      packet += (char)LoRa.read();
    }
    Serial.print("Received packet: ");
    Serial.println(packet);

    // Check if it's a flip alert message
    if (packet.indexOf("FLIP") != -1) {
      int idPos = packet.indexOf("ID");
      char senderID = (idPos != -1 && idPos+2 < packet.length()) ?
packet.charAt(idPos+2) : '?';
      Serial.print(">>> Boat ");
      Serial.print(senderID);
      Serial.println(" FLIPPED! <<<");
      // (Here you could activate a buzzer or LED to alert the crew.)
    }
  }

  delay(100);  // small delay to limit loop speed and allow serial print flush
}
```

Using the code: Update the `#define BOAT_ID` at the top to **1** for the first boat's Arduino, upload the code, then change it to **2** and upload to the second boat's Arduino. Both will then run the same logic. You can test the system by physically inverting one MPU6050 (simulating a flipped boat) and observing the Serial output on the other Arduino – it should print a flip alert identifying the correct boat ID.

Summary and Further Considerations

By following these steps, we created a basic two-boat communication system: - Each boat continually monitors its orientation via the MPU6050 sensor. When the tilt exceeds a threshold (indicating a capsize), it sends a LoRa radio message. - The LoRa modules are configured for direct radio packet communication (point-to-point). We defined a simple message protocol (`"IDx:FLIP"`) so that a receiver can know which boat sent the alert. - We ensured each boat has a unique ID (set in code or via hardware) and took care to wire the hardware correctly (powering the LoRa module at 3.3V [1] and using the proper SPI connections [15] [3], with I2C for the MPU6050 [7]).

This example can be extended or modified as needed. For instance, you might add an LED or buzzer that activates when a flip message is received, or implement an acknowledgment so the flipped boat knows the message was received. You could also expand the packet format to handle other events or sensor data. But

as a foundation, this guide provides a clear step-by-step approach to achieve the core functionality – two Arduinos with LoRa radios notifying each other of a flip-over event.

Sources: The wiring and LoRa setup are based on typical SX1278–Arduino integrations [15] [3], using Sandeep Mistry's LoRa library [9]. The orientation calculation from accelerometer data relies on standard formulas for roll/pitch derived from gravity vectors [12]. The importance of using 3.3V and proper antenna for the LoRa module is emphasized in documentation [1] [6]. The code structure (parsePacket, beginPacket, etc.) follows the examples in the LoRa library [13] [14].

---

[1] Interfacing SX1278 (Ra-02) LORA Module with Arduino - Hackster.io
https://www.hackster.io/shathiralakdilu/interfacing-sx1278-ra-02-lora-module-with-arduino-db8e1f

[2] [3] [4] [5] [6] [9] [10] [13] [14] [15] Arduino LoRa Tutorial: Interfacing SX1278 (Ra-02) LoRa Module with Arduino
https://circuitdigest.com/microcontroller-projects/arduino-lora-sx1278-interfacing-tutorial

[7] [8] Arduino | MPU6050 6-DoF Accelerometer and Gyro | Adafruit Learning System
https://learn.adafruit.com/mpu6050-6-dof-accelerometer-and-gyro/arduino

[11] [12] Roll and Pitch Angles From Accelerometer Sensors - Michael Wrona's Blog
https://mwrona.com/posts/accel-roll-pitch/